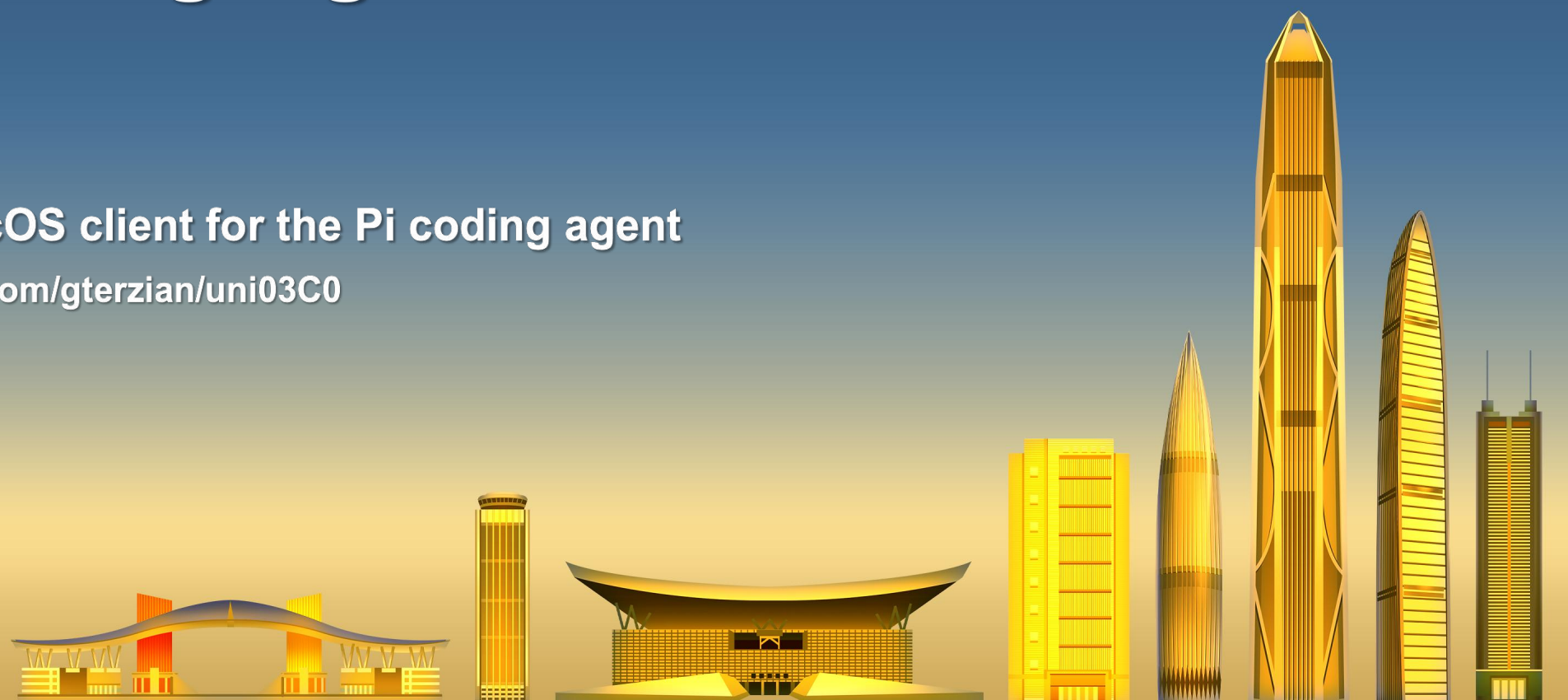
GOSIM

GOSIM Shenzhen 2026

Usable Coding Agent UI

uni03C0: a native macOS client for the Pi coding agent

Gregory Terzian | github.com/gterzian/uni03C0



My experience with TUI agents

1

Content shifts

Output streaming in below the viewport changes the text above it, including when the reader has scrolled up. Reading and copying a passage becomes hard.

2

Paste stalls

Pasting a very large text, such as a complete Lean proof, blocks the UI and shows no progress.

3

CPU above 100%

A session beyond 500k tokens of context holds the CPU above 100% in Activity Monitor while output streams in.

4

No sandbox

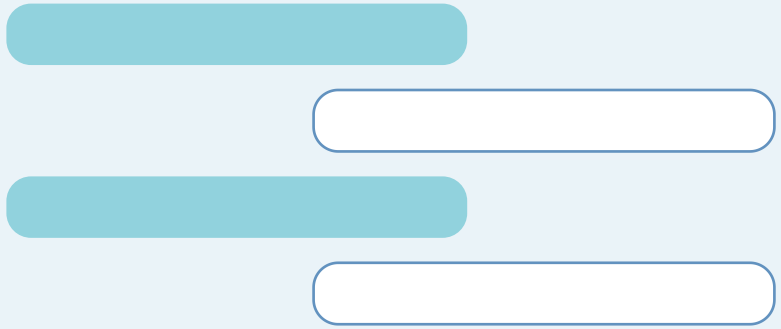
The agent runs in yolo mode with full read and edit access to the user's files.

Observed with the Pi TUI in the macOS terminal (Pi issue 7730)



Coding sessions are continuous streams

Chat



- Short messages, a few lines each
- Updates arrive when someone sends a message
- The screen settles between messages

Coding session



- Text streams in without stopping
- Thinking, tool calls, file reads, diffs and command output
- Context windows above 500k tokens
- A single tool output can be very large

A different approach: three ideas

Never block the UI

Decoding and search run off the main thread.

Idea 1

Only render what you need

Only what is on screen gets drawn.

Idea 2

Native sandbox

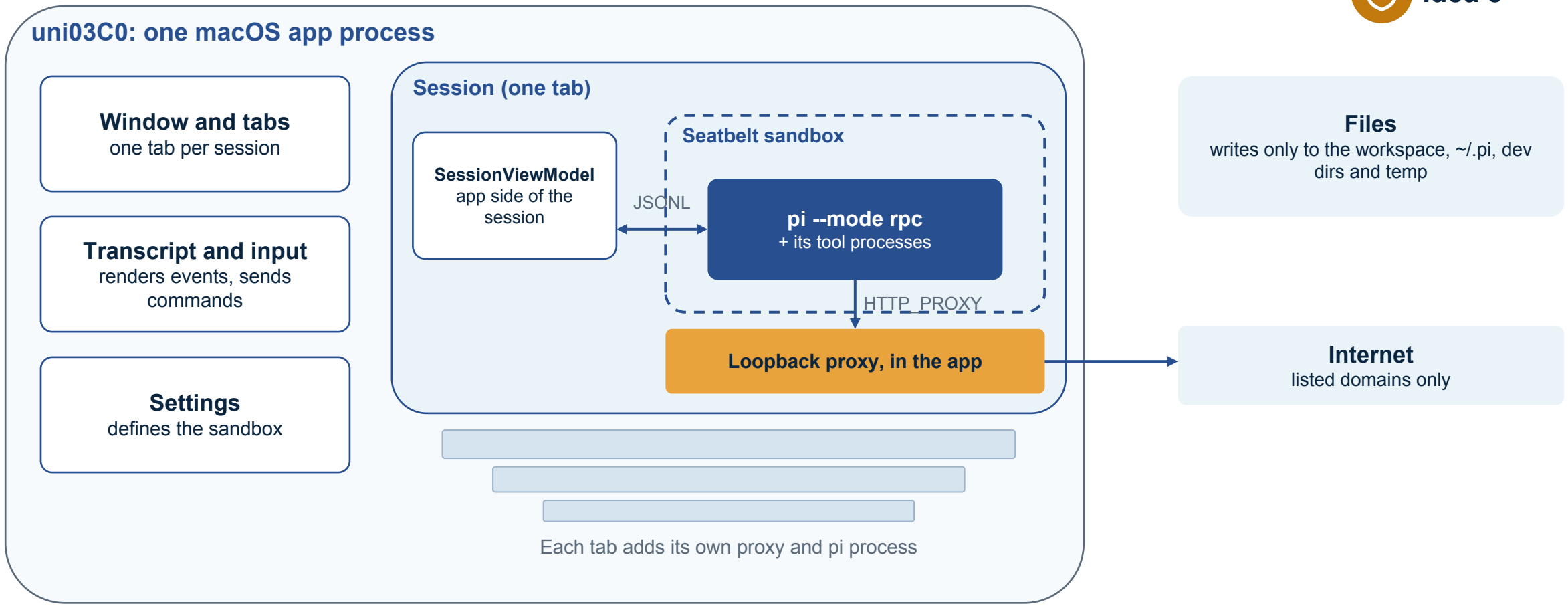
Each agent runs in a sandbox built on native macOS technology.

Idea 3



Architecture: app and pi processes

Idea 3



Each session gets its own pi process, proxy and sandbox. Settings are captured at session start.

Sandbox: setup and enforcement

Idea 3

Setup on first run

1 Working folder
read + write

→

2 Extra paths
toolchains, caches, Xcode

→

3 Internet domains
subdomains included

→

4 Session start
settings captured

What enforces it at runtime

Seatbelt sandbox: denies by default

Launcher
sandboxes itself

exec →

Agent
inherits the policy

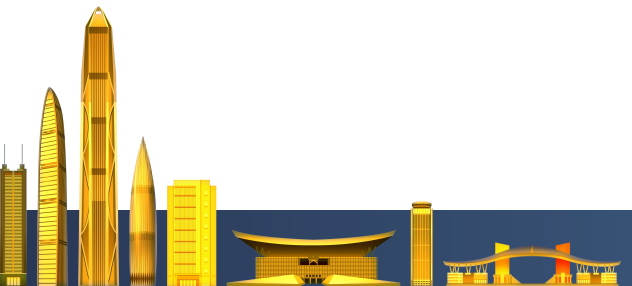
proxy →

Domain proxy
runs unsandboxed in the app

→

Listed domains only
reached through the proxy

Everything not listed is denied: files, network and system calls.



Session: work off the main thread

Idea 1

TranscriptStore

Worker: Core, off the main thread

- Holds the full history
- Decodes, applies and searches

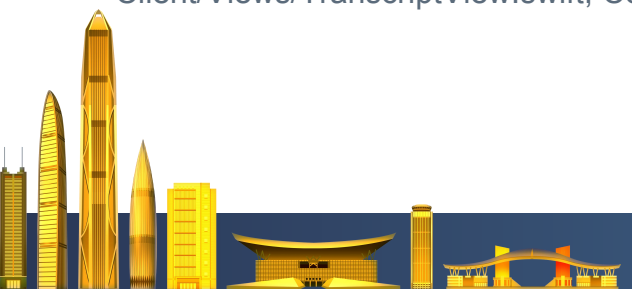
Coordinator and table

Main thread: Client and AppKit

- Pulls only what is on screen
- Draws the visible rows

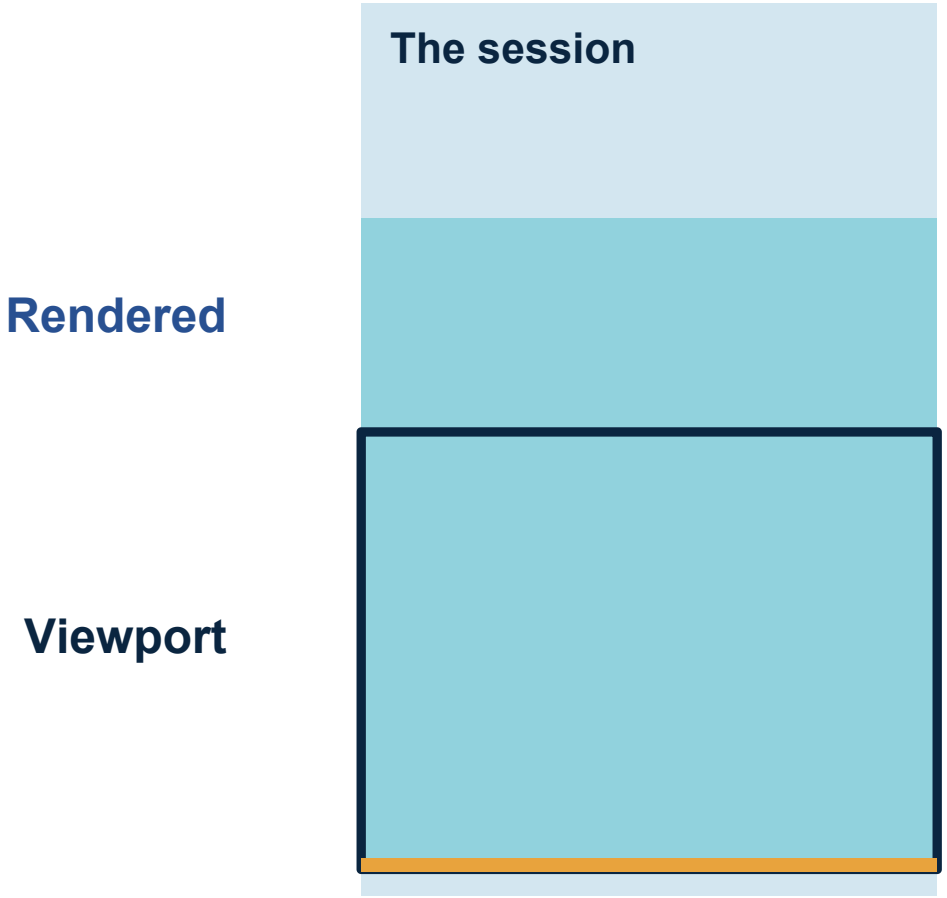
Decoding, applying and searching run on the worker. The main thread draws the visible rows.

Client/Views/TranscriptView.swift, Core/Session/TranscriptStore.swift



Session: draw the viewport

Idea 2



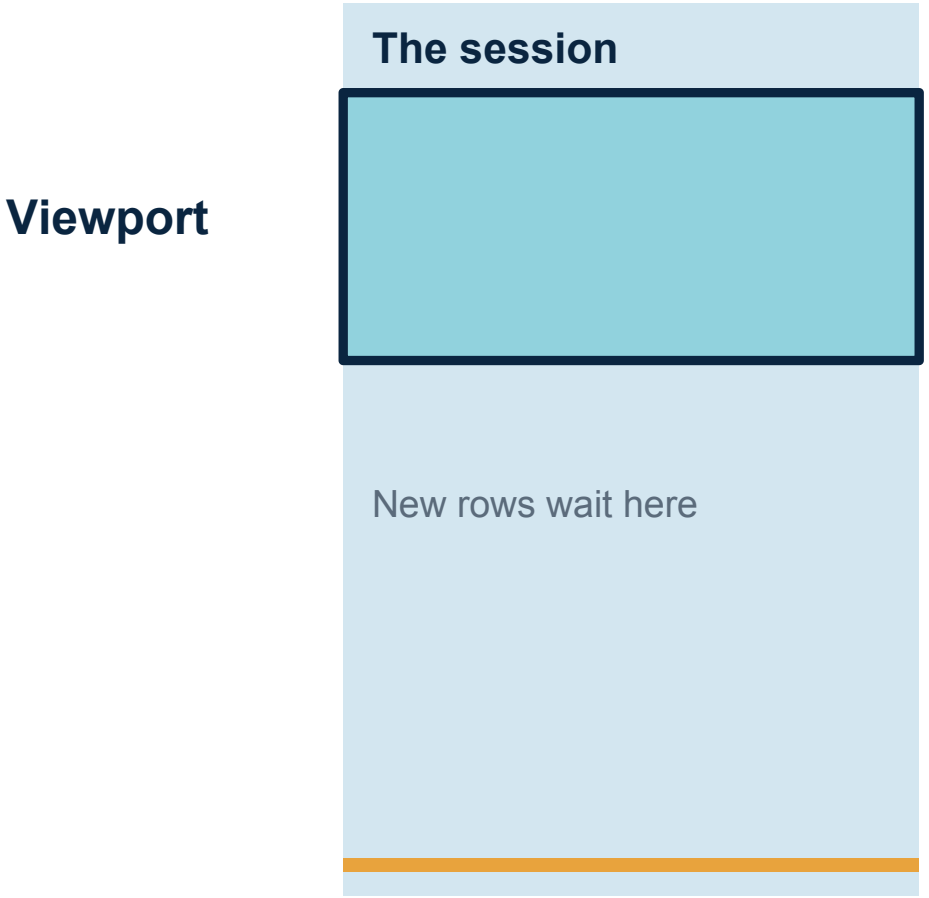
When the viewport includes the tail: re-render as content streams in.

When scrolled away from the tail: the view stays unchanged.

The table keeps the rows on screen plus about three screens above them.

Scrolled up: new rows wait in the store

Idea 2



When scrolled away from the tail: the view stays unchanged.

New rows go into the store.

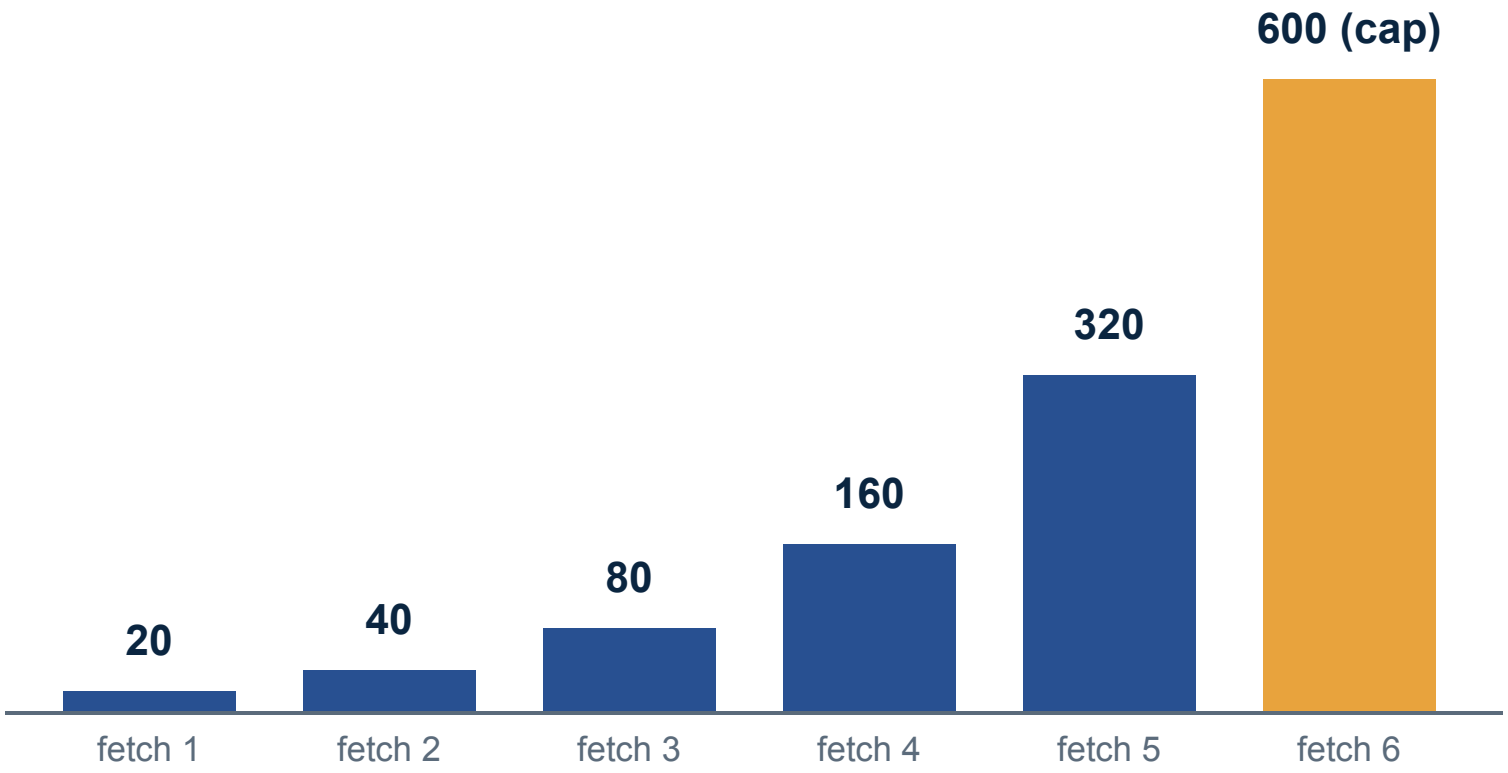
Only the rows on screen are drawn.

Scrolling back to the tail draws the new rows.

Scrolling further up: history stays in memory

Idea 2

Rows loaded per step (first step: 20 or more)



Client/Views/TranscriptView.swift: fetchBlock, fetchBlockMax

- Older blocks load in doubling steps, up to 600 rows
- The store keeps the full history, so reloads are instant
- Scrolling down drops the rows above the buffer again
- Cmd+Up and Cmd+Down jump between your user messages

Elsewhere: paste and diffs, one window

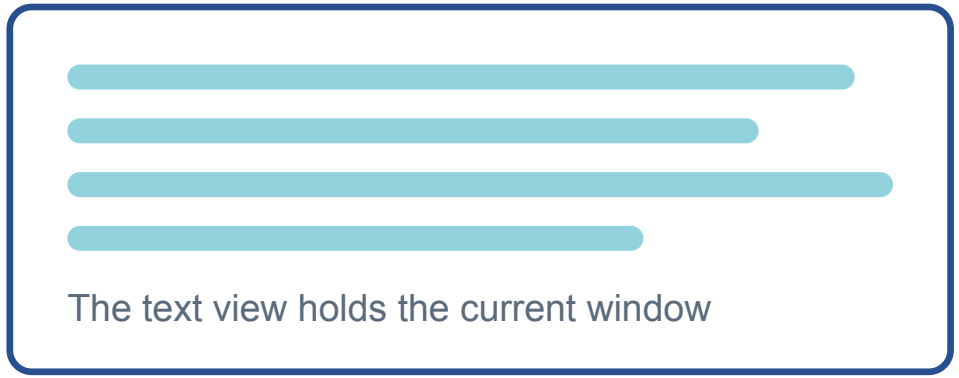
Idea 2

A large paste or diff is laid out one window at a time.

Store: the full paste, or the full changed file

Window

Paste: 65,536 UTF-16 units. Diff: changed hunks with context.



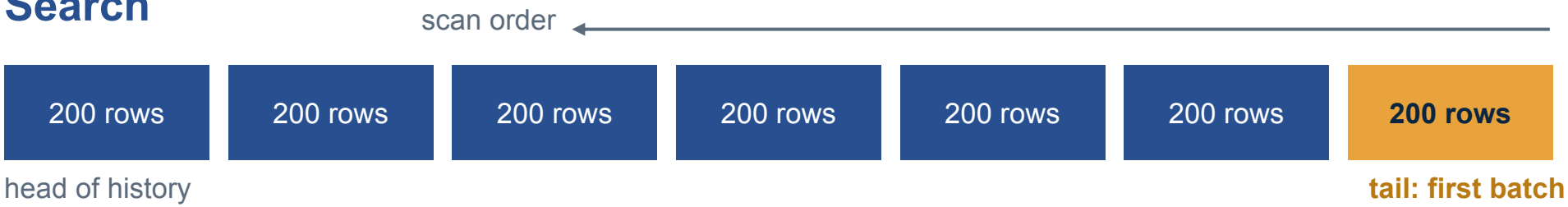
slides toward the head or the tail with the scroll position

- Layout cost follows the window size
- The tail of a paste appears first
- Diffs collapse unchanged gaps into expand controls
- Copying a diff selection creates a reference: file, line range and snippet

Elsewhere: batches make work abortable

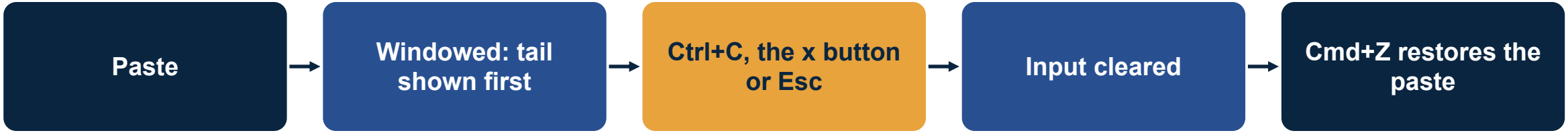
Idea 1

Search



- Each slice is searched in its own task, from the tail upward, under a read lock
- A new keystroke cancels the run at the next batch boundary; results land as each batch completes

Large paste



A multi-megabyte paste keeps the UI responsive and one key removes it. Esc also stops the running turn.

Search: session and diff view

Session search

Idea 1

Cmd+F
opens the find bar

→

Type
search starts after a 500 ms pause

→

Scan
batches of 200 rows, from the tail up

→

Results
newest match first; Enter cycles

Diff view search

Idea 2

Cmd+F
opens the find bar

→

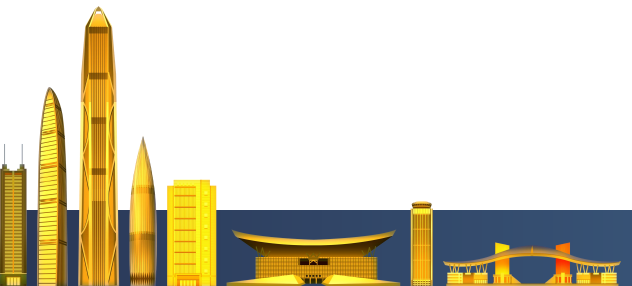
Type
one scan of the whole diff buffer

→

Highlight
all hits marked, current hit shown

→

Jump
first hit at or below the top; Enter cycles



GOSIM

GOSIM Shenzhen 2026

Thanks!

github.com/gterzian/uni03C0

medium.com/@polyglot_factotum



GOSIM

GOSIM Shenzhen 2026

Thanks!

github.com/gterzian/uni03C0

medium.com/@polyglot_factotum



GOSIM

GOSIM Shenzhen 2026

Thanks!

github.com/gterzian/uni03C0

medium.com/@polyglot_factotum

